

Name: Garv Arora | Reg. No: 23BCT0005 | BCSE497J|

ARIA

Autonomous Reasoning & Interaction Agent

A cognitive, multi-agent vision-language-action software stack for adaptive manipulation planning.

Research Report

1. Project Title

ARIA — Autonomous Reasoning & Interaction Agent. A cognitive, multi-agent vision-language-action software stack for adaptive manipulation planning.

2. Abstract

ARIA is a locally-run, multi-agent manipulation system built around an affordable 5-DoF arm. It understands spoken or typed instructions, works through them step by step, and picks up, moves, and manipulates objects roughly the way a person would: it adjusts for uncertainty, checks in with the user when it isn't sure, and keeps getting better at its own skills over time.

3. Full Project Description

ARIA runs on the simulation of a 5-degree-of-freedom arm: SG90 simulated servos handle the lighter joints, and MG995 simulated servos units take the load-bearing ones. Vision comes from either an ESP32-CAM simulation or a Logitech C270 like camera node, and an MPU6050 IMU simulated sensor sits on the end effector to supply orientation feedback the camera alone can't give. Commands arrive as natural language and get broken down through a fixed hierarchy: command, goal, subgoals, actions. Every step is written out as a plain-language chain of thought, for example locate the object, locate the target, plan the grasp, execute it, verify it, move, release, and confirm completion.

Perception is deliberately hybrid rather than purely learned. YOLO handles fast per-frame detection, while monocular depth from Depth-Anything or MiDaS is combined with straightforward camera and table-height calibration geometry to produce a reliable Z coordinate. That combination matters because there's no dedicated depth sensor anywhere in this setup. Grasp candidates come from GraspNet or AnyGrasp and then get filtered by an Affordance Agent, which tries to pick the grasp that matches how an object is actually used rather than just the nearest reachable surface. Motion runs through whichever inverse-kinematics solver tests out best for the arm's geometry (ikpy, roboticstoolbox-python, an analytical solution, trac-ik, KDL, or a neural IK model), executed through MoveIt2 with cubic or quintic spline trajectories, and planned with awareness of both workspace obstacles and the arm's own trailing cables.

Decisions come from an LLM-based Planning Agent, ideally run locally, using function-calling to hand work off to the right specialist agent. Optional Tree-of-Thought or Monte Carlo Tree Search exploration kicks in when a task has real branching complexity. Every action carries a confidence score, and when that confidence drops low enough, the system stops and asks the user before proceeding instead of guessing through uncertainty silently. A World Model Agent keeps a running semantic map of known objects, including their coordinates, color and material attributes, and a lifecycle state that moves through detected, tracked, lost, recovered, moved, and removed. If an object isn't where the map says it should be, ARIA checks with the user rather than assuming it simply failed to see it.

Underneath all of that, the system keeps watching its own health (servo temperature, mechanical drift, calibration drift, camera frame rate, inference latency) and its own performance (localisation error, IK error, pick success rate, task completion rate, planning and inference time, recovery rate). That data feeds a Learning Agent, which gradually improves a fixed library of skills (pick, place, push, pull, stack, sort, inspect, slide, roll, sweep), and an Evaluation Agent, which sorts failures into useful categories: missed object, object slipped, IK failure, collision, or perception error, so troubleshooting has somewhere to start. A separate learn mode lets someone teleoperate the arm directly, logging images, joint states, and commands for later use in improving autonomous behavior.

4. System Architecture and Execution Flow

The diagram below traces a single command from the moment it's spoken through to the arm actually moving, then back through the monitoring and learning loop that feeds improvements back into planning.

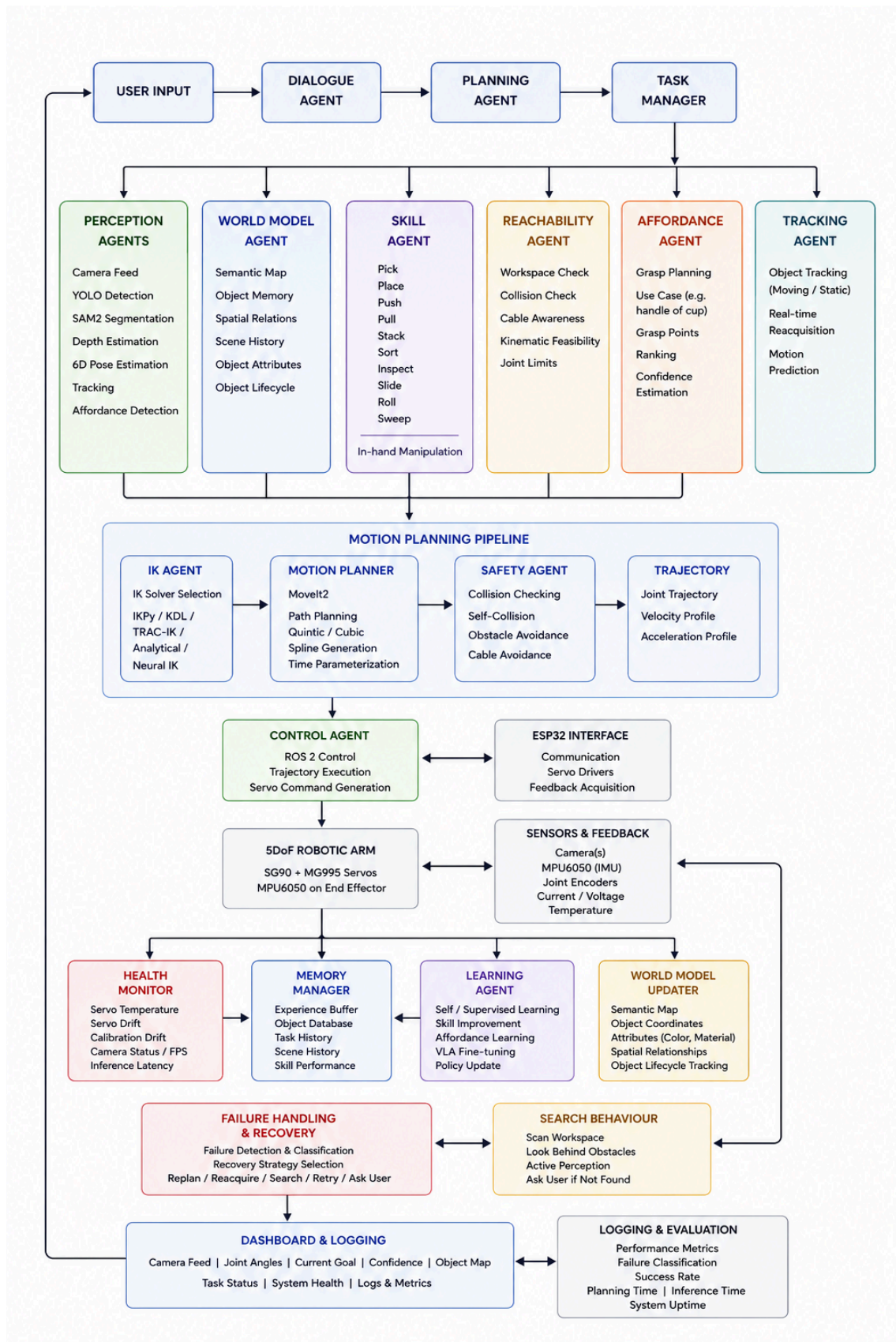


Figure 1. ARIA flow diagram

5. Where the Novelty Actually Is

None of the individual pieces here are new on their own. YOLO, depth estimation, GraspNet, MoveIt2, and open VLA policies like LeRobot or OpenVLA already exist and get used across robotics research. What this project actually proposes is a particular way of wiring all of it together, so that a \$150–200 arm can behave like something closer to a research-grade platform, done with an architecture that stays legible enough to debug. A few things make the combination worth calling out specifically.

- **Agent-hierarchy scale.** Around fifteen cooperating agents (vision, depth, tracking, affordance, planning, skill, control, safety, memory, world model, learning, evaluation, dialogue, attention, reachability) run together on a single consumer GPU, which most published multi-agent robotics stacks don't attempt.
- **Hybrid perception.** Classical camera and table-height calibration geometry combines with learned monocular depth to recover a trustworthy Z coordinate, useful when the only camera on hand is an ESP32-CAM or a webcam with no native depth sensing.
- **Affordance-and-confidence loop.** Grasps get chosen based on how an object is actually used, a cup by its handle, a tool by its shaft, and the system only pauses to ask the user when its own confidence estimate says it should.
- **Self-supervised operational logging.** IK error, pick success rate, recovery rate, and localisation drift feed straight into a Learning Agent that improves the skill library over time.
- **Explainability.** An explicit chain-of-thought log runs for every task, alongside cable-aware, self-calibrating motion planning on hobby servos, neither of which is common at this price point.

Taken individually, each of these is incremental. Put together as a system, though, they add up to something that behaves noticeably more capable than the sum of its parts would suggest.

6. Use Cases

- **Research and teaching.** A way for students and small labs to work with embodied AI, multi-agent coordination, and vision-language-action models without needing an industrial arm or an expensive RGB-D rig.
- **Desktop and lab automation.** Sorting small parts, organizing tools, handling lab consumables that don't need much force.
- **Assistive-manipulation prototyping.** Testing whether an arm can fetch everyday objects like cups, bottles, or tools on a spoken command, before committing to more expensive hardware.
- **Benchmarking testbed.** Running the same task across different IK solvers, depth models, or VLA policies to see which actually performs best on identical hardware.
- **Human-robot interaction research.** The confidence-gated approval workflow and chain-of-thought logging make this a decent platform for studying trust and explainability.
- **Dexterity research.** In-hand manipulation goals such as rotating a screwdriver or repositioning a paintbrush give it a natural angle into fine manipulation.

7. Benefits

- **Lower cost of entry.** SG90/MG995 servos and a webcam instead of an industrial arm and a dedicated depth camera bring the price down by an order of magnitude or more.
- **Explainability.** Chain-of-thought logging makes every decision traceable, which helps with debugging and is genuinely useful for teaching.
- **Safer operation.** The confidence-gated approval step and a dedicated Safety Agent reduce the odds of damaging an object or the arm itself.
- **Continuous improvement.** Logged success rate, IK error, and recovery rate give the Learning Agent something concrete to work from, without a full retraining cycle.
- **Modularity.** The clean separation between the arm, perception, and planner layers means the same stack could, in principle, be pointed at a different arm with only modest rework.
- **Graceful degradation.** The system actively looks for better viewing angles, searches the workspace when something's missing, and tries to recover from failures rather than just stopping.

8. Tech Stack

Everything ARIA is actually built from and runs on, split from the AI models and algorithms in the next two sections.

Category	Component	Purpose
Mechanical	5-DoF robotic arm frame	Base mechanical platform for the whole system
Actuation	SG90 micro servos	Lightweight, lower-torque joints
Actuation	MG995 servos	Load-bearing joints that need more torque
Vision sensor	ESP32-CAM or Logitech C270	Primary camera feed for detection and depth
Orientation sensor	MPU6050 IMU	End-effector orientation feedback the camera can't provide alone
Firmware / link	ESP32 Interface	Bridges the ROS2 stack to the servo hardware
Compute	Lenovo LOQ · Intel i7-13700HX · RTX 5060	Runs perception, VLA, and LLM planning locally
Middleware	ROS2	Node and topic backbone connecting every package
Motion framework	MoveIt2	Hosts collision-aware motion planning and execution
LLM runtime	Ollama	Local serving for the planning LLM (Llama 3.1 8B, by default)
Calibration hardware	AprilTag fiducials	Physical markers used for camera-to-arm self-calibration

9. AI Models in Use

Every learned model in the stack, what it's responsible for, and why it was picked over the alternatives. Classical solvers and search algorithms are kept separate, in the next section.

Model	Role	Why it's used
YOLO (v8/v9)	Object detection	Fast enough to run every frame; gives the Vision Agent its first pass at what's in view.
SAM2	Instance segmentation	Pixel-accurate masks instead of boxes, which helps with grasp region estimation and partial occlusion.
Depth-Anything	Monocular depth	Primary depth source when there's no stereo or RGB-D camera to fall back on.
MiDaS	Monocular depth (alternate)	Benchmarked directly against Depth-Anything; whichever performs better under the actual lighting wins.
GraspNet	Grasp pose generation	Proposes candidate grasps directly from point-cloud or RGB-D input for objects it hasn't seen before.
AnyGrasp	Grasp pose generation (alternate)	Compared against GraspNet on grasp success rate and speed.
LeRobot	Vision-language-action policy	Open framework mapping images and instructions directly to actions; used for the learned manipulation skills.
OpenVLA	VLA policy (alternate)	Benchmarked against LeRobot on instruction-following accuracy and inference speed.
Pi0	VLA policy (alternate)	A flow-based policy tested as a third option for general manipulation behavior.
GR00T N1	VLA policy (alternate)	Foundation-model-based option, heaviest of the four, evaluated for transfer of manipulation skill.
Neural IK	Inverse kinematics (learned alternate)	A learned solver benchmarked against the classical IK options for accuracy and solve time on this geometry.
Llama 3.1 8B (or GPT-4o / Qwen2.5-VL)	Natural-language planning	Parses ambiguous multi-step commands and dispatches work through function-calling; run locally where possible.
FoundationPose / MegaPose	6D pose estimation (planned)	Adds full orientation on top of position, needed for tasks like key insertion.
ClearGrasp / TransCG	Transparent-object depth (planned)	Corrects depth errors on glass and reflective surfaces where standard depth models fail.
3D Gaussian Splatting	Scene reconstruction (planned)	Builds a persistent 3D reconstruction of the workspace instead of recomputing depth every frame.

10. Algorithms in Use

Algorithm	Role	Why it's used
ikpy / roboticstoolbox-python / trac-ik / KDL / analytical IK	Inverse kinematics	Benchmarked against each other and against Neural IK on accuracy, solve time, and singularity handling for this specific 5-DoF geometry.
MoveIt2	Motion planning and execution	Handles collision-aware path planning and ties execution to the trajectory generator.
Cubic / quintic splines	Trajectory generation	Smooths joint or Cartesian paths into motion the servos can actually follow without jerking.
Camera & table-height calibration geometry	Z-coordinate recovery	Converts a detected pixel into a real-world table coordinate using known camera and table height, without needing a depth sensor.
Tree-of-Thought / Monte Carlo Tree Search	Plan search	Explores multiple candidate plans for complex tasks and picks the one most likely to succeed.
AprilTag detection	Self-calibration	Fiducial markers used to keep the camera-to-arm calibration accurate over time.

11. Agents in the System

Agent	Responsibility
Vision Agent	Object detection and segmentation from the live camera feed (YOLO, SAM2).
Depth Agent	Monocular depth estimation combined with calibration geometry.
Tracking Agent	Following moving objects in real time, a rolling pencil, for instance.
Affordance Agent	Choosing the grasp that matches how an object is actually used.
Planning Agent	Breaking a command down into goals, subgoals, and concrete actions.
Skill Agent	Running and gradually improving the predefined skill library.
Control Agent	Turning motion plans into low-level servo commands.
Safety Agent	Collision avoidance and awareness of the arm's own cables.
Memory Agent	Keeping a record of past objects, tasks, and outcomes.
World Model Agent	Maintaining the semantic map: object coordinates and lifecycle state.
Learning Agent	Improving skills over time from logged, self-supervised data.
Evaluation Agent	Classifying failures and tracking success-rate and error metrics.
Dialogue Agent	The natural-language interface, including clarification and approval prompts.
Attention Agent	Directing the camera to a better angle when the current view isn't good enough.
Reachability Agent	Checking whether a target actually falls inside the arm's workspace.
Health Monitoring	Watching servo temperature, drift, camera frame rate, and inference latency.

12. Future Scope

- **6D pose estimation** (FoundationPose or MegaPose): adds orientation awareness for tasks like inserting a key or connecting a plug, something the current X/Y/Z-only pipeline can't do.
- **SAM2 segmentation** in place of YOLO's bounding boxes: pixel-accurate masks should improve grasp planning and handling of partially occluded objects.
- **3D Gaussian Splatting** reconstruction of the workspace, letting the system reason about the scene persistently, without re-running a full depth pass every frame.
- **Material recognition** feeding directly into grasp force, so the arm grips glass gently and a metal tool firmly.
- **Containerized deployment**, CI/CD with simulated regression tests, and experiment tracking (Weights & Biases or similar), to make the system easier to maintain and compare against itself as it changes.
- **Scaling up** from one arm to a multi-arm or mobile-base setup, backed by a growing database of known objects with their CAD models, materials, and learned affordances.

13. Drawbacks and Honest Limitations

- **Hardware ceiling.** SG90 and MG995 servos have backlash, drift, and limited torque compared to industrial actuators, and no amount of software will fully compensate for that.
- **Compute risk.** Running VLA models, depth estimation, segmentation, and an LLM planner all at once on a single RTX 5060 will need careful scheduling and probably some model quantization to stay within real-time budgets.
- **Sensing limits.** A webcam or ESP32-CAM paired with monocular depth is less accurate than a proper stereo or RGB-D setup, especially on transparent, reflective, or very thin objects.
- **System complexity.** Fifteen-plus cooperating agents, several IK solver options, several VLA backends, and two depth models add up to a lot of surface area for a small team to keep working correctly at once.
- **Latency stacking.** By the time an image has gone through detection, depth, grasp planning, IK, and trajectory generation, the accumulated delay may be too slow for something genuinely fast-moving, like a rolling pencil.
- **No dedicated force sensor.** Force estimation is inherently approximate, which is a real limitation for anything fragile or high-force.